



Universidad Nacional Experimental de Guayana

Vicerrectorado Académico

Proyecto de Carrera: Ingeniería en Informática

Unidad Curricular: Lenguajes y Compiladores

FASES DEL COMPILADOR Y SUS TENDENCIAS

LOS ASTRONAUTAS

"EXPLORANDO EL UNIVERSO DE LOS LENGUAJES, UN NODO A LA VEZ"

PROFESOR:
FÉLIX MÁRQUEZ

PRESENTADO POR:

C.I.: V-25.933.680. ALBURQUERQUE SHEEN

C.I.: V-30.907.427. ANTOIMA MARIANGEL

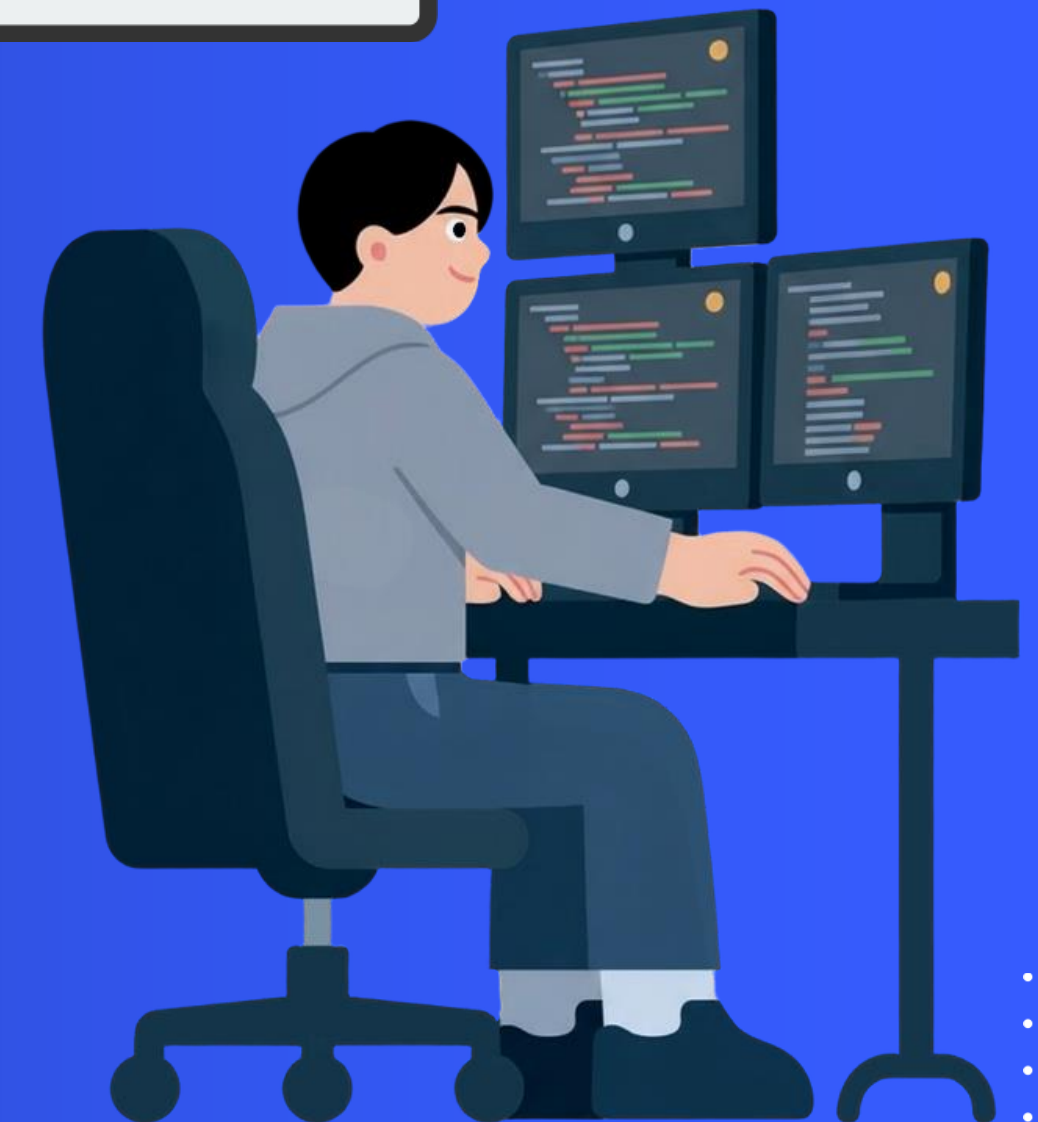
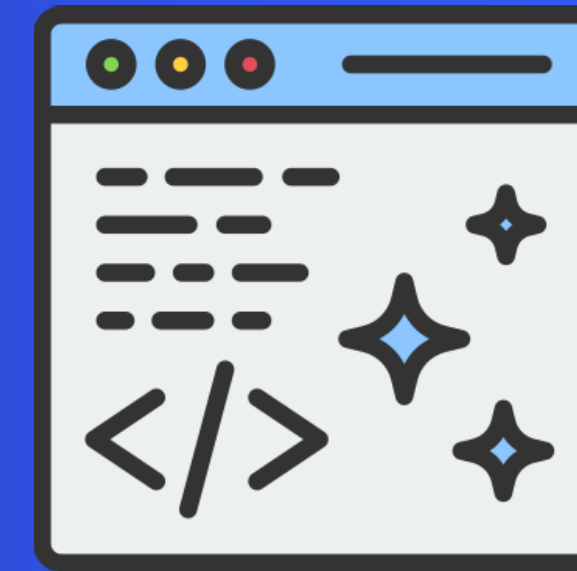
C.I.: V-28.475.271. GARCÍA CARLOS

C.I.: V-24.848.424. VARGUILLAS GÉNESIS

FUNDAMENTOS DE LA COMPILACIÓN

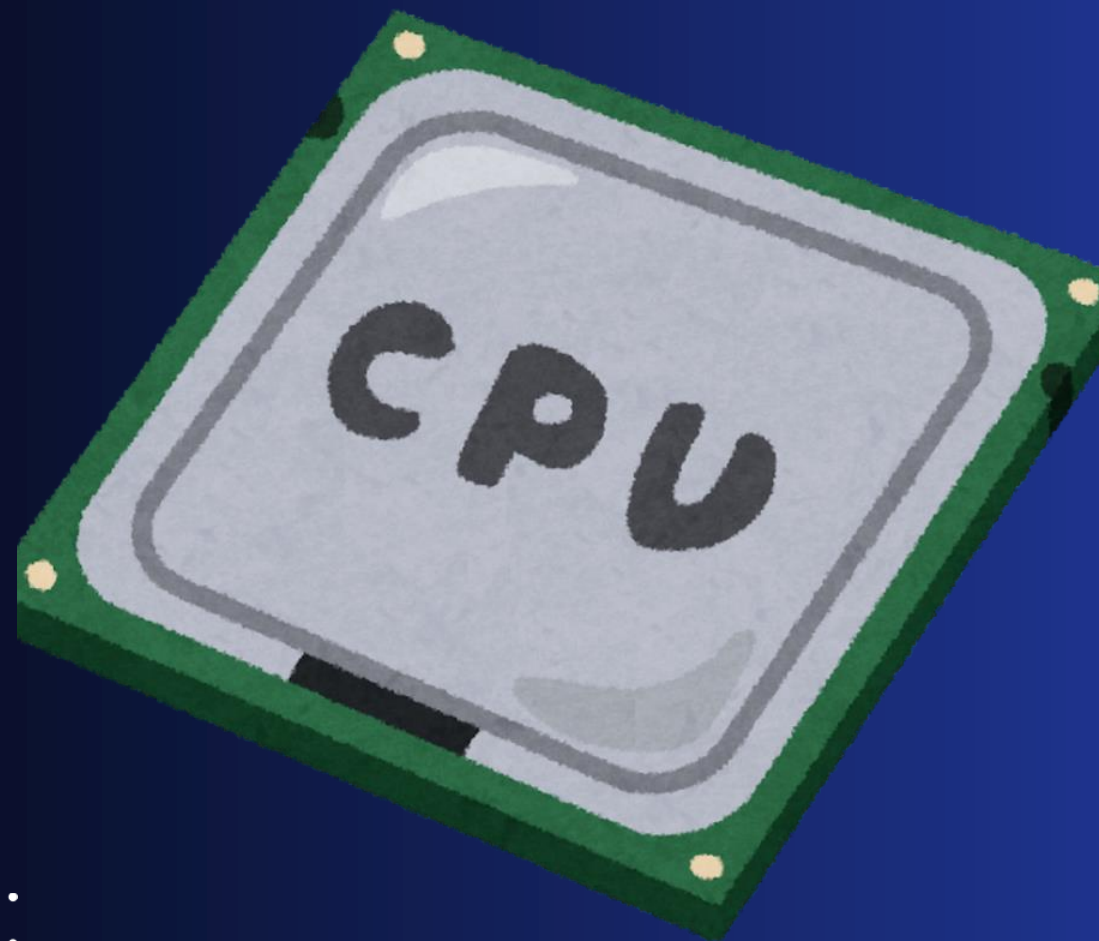
UN COMPILADOR ES UN PROGRAMA QUE TRADUCE CÓDIGO FUENTE ESCRITO EN UN LENGUAJE DE ALTO NIVEL A CÓDIGO MÁQUINA.

Además de traducir, también analiza errores y optimiza el programa.



COMPILADOR

- TRADUCE EL PROGRAMA COMPLETO
- Genera un archivo ejecutable
- Mayor velocidad de ejecución



vs

INTÉRPRETE

- TRADUCE LÍNEA POR LÍNEA
- No genera ejecutable
- Mayor flexibilidad



OBJETIVOS DEL COMPILADOR



CORRECCIÓN

EL PROGRAMA TRADUCIDO DEBE COMPORTARSE EXACTAMENTE COMO FUE DISEÑADO.



EFICIENCIA

EL COMPILADOR BUSCA GENERAR CÓDIGO RÁPIDO Y OPTIMIZADO PARA APROVECHAR MEJOR LOS RECURSOS DEL SISTEMA.

LOS COMPILADORES MODERNOS INCLUYEN DIFERENTES NIVELES DE OPTIMIZACIÓN.

EVOLUCIÓN HISTÓRICA DE LOS COMPILADORES



LA EVOLUCIÓN DE LOS COMPILADORES TRANSFORMÓ LA PROGRAMACIÓN MODERNA.

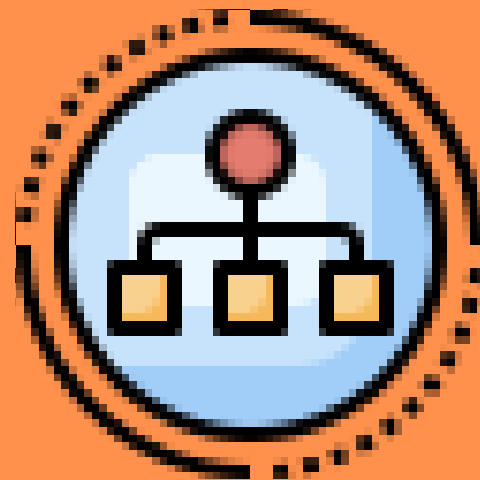
ANATOMÍA DEL COMPILADOR

LAS FASES DEL PROCESO DE COMPILACIÓN

El Front-End analiza el código fuente para asegurar que cumpla con las reglas del lenguaje antes de su traducción.



Análisis Léxico
Lee y tokeniza



Análisis Sintáctico
Verifica estructura



Análisis Semántico
Comprueba significado

ANÁLISIS LÉXICO (SCANNER)



El lexer lee el código carácter a carácter y agrupa los caracteres en tokens: las "palabras" del compilador.

- Reconoce **if, while, variables, números, operadores**
- Elimina espacios, tabulaciones y comentarios
- Cuenta líneas y columnas para reportar errores con precisión
- Asocia tipo y valor a cada token

ANÁLISIS LÉXICO (SCANNER)

HERRAMIENTAS DEL LEXER

Nadie escribe un Lexer desde cero. Se usan generadores:

- **Flex:** El estándar en Unix/Linux.
- **Lex:** La herramienta clásica original.
- **JLex:** Versión para ecosistemas Java.

EJEMPLO: $X = 3 + 5$

TOKEN_ID("x")

TOKEN_OP_ASIG("=")

TOKEN_NUM(3)

TOKEN_OP_ARIT("+")

TOKEN_NUM(5)



ANÁLISIS SINTÁCTICO (PARSER)

El parser verifica que la secuencia de tokens cumpla con la gramática del lenguaje, definida mediante **Gramáticas Libres de Contexto (GLC)**.

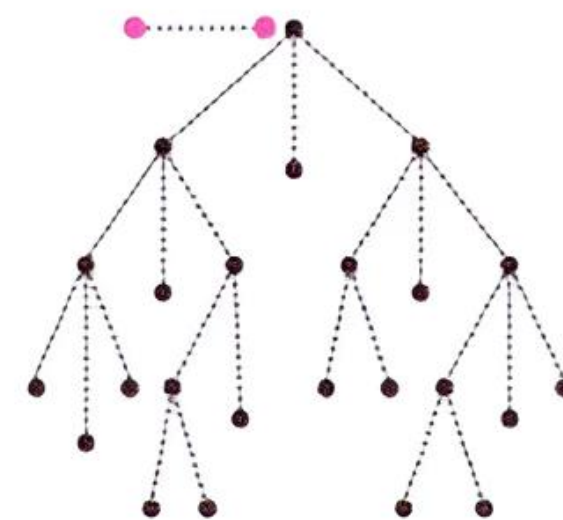
Una GLC define: símbolos terminales (tokens), no terminales (categorías), producciones y un símbolo inicial.



Herramientas: **Bison** (C) y **JavaCUP** (Java)
generan parsers desde una descripción gramatical

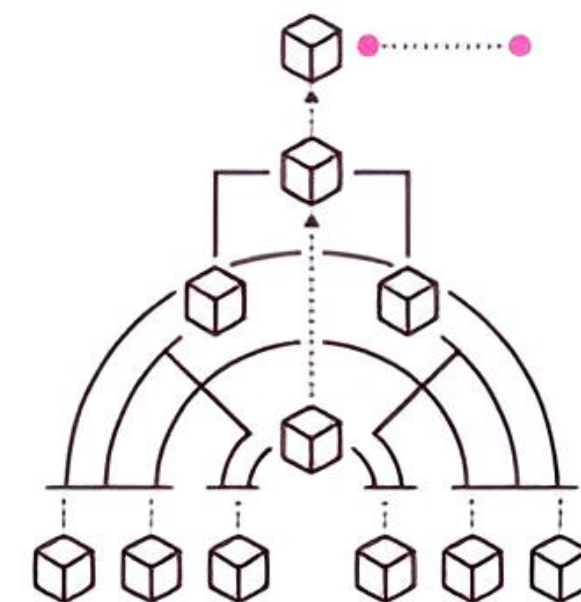
TIPOS

PARSERS DESCENDENTES LL



Parten del símbolo inicial hacia los tokens. Fáciles de implementar manualmente (ej: Recursivo Descendente, LL(1)).

PARSERS ASCENDENTES LR



Parten de los tokens hacia el símbolo inicial. Más potentes y complejos (ej: LALR, SLR, LR(0), LR(1)).

EL AST: NUESTRA SEÑA DE IDENTIDAD

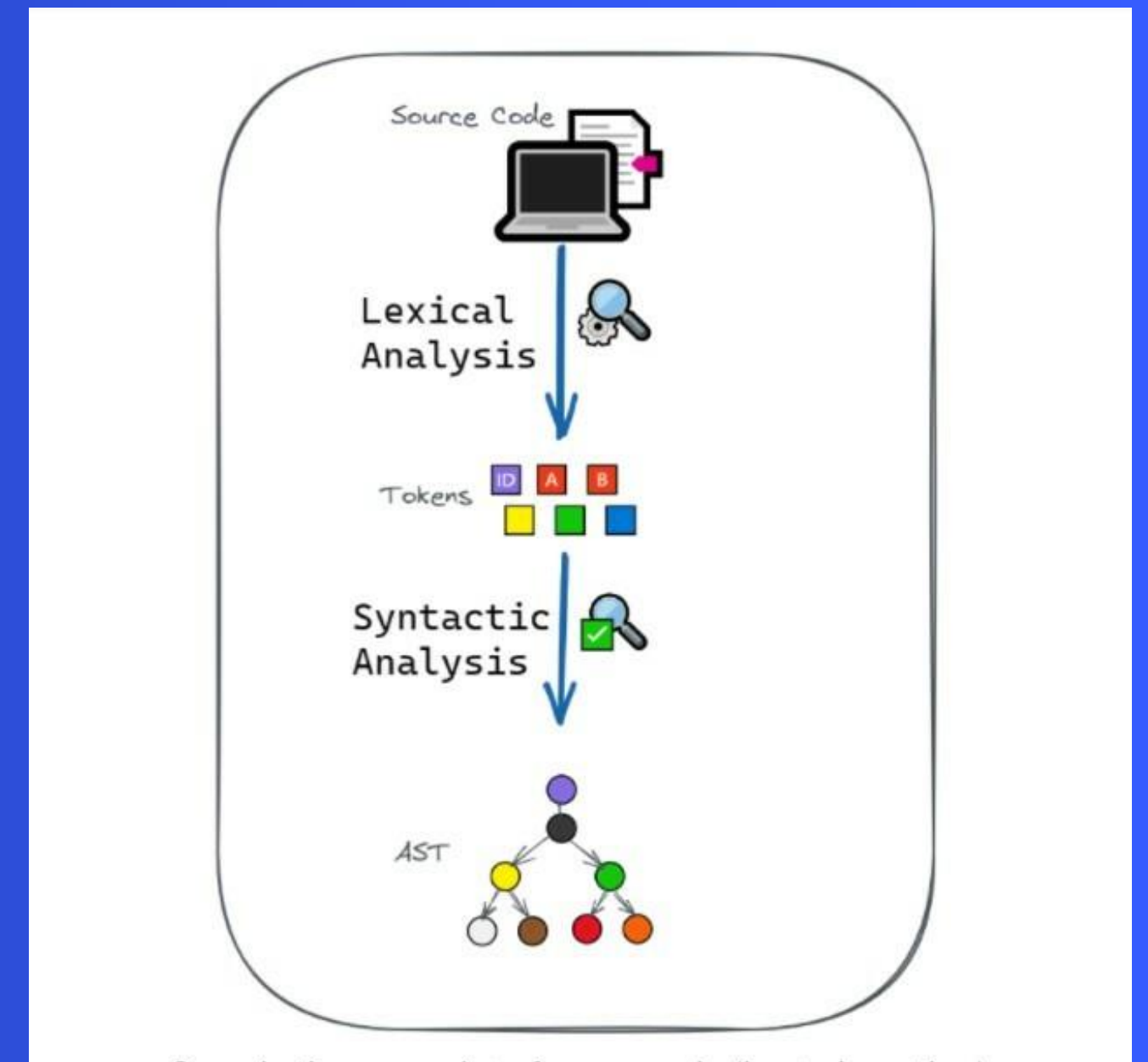
El parser construye un Árbol Sintáctico Abstracto (AST): una representación compacta que elimina detalles superfluos y muestra cómo el compilador "piensa" el código.

Árbol Sintáctico

Representación completa pero con información redundante (paréntesis, punto y coma, nodos intermedios).

AST

Versión simplificada.
En $a + b * c$, el árbol muestra claramente que $*$ tiene prioridad sobre $+$.



ANÁLISIS SEMÁNTICO

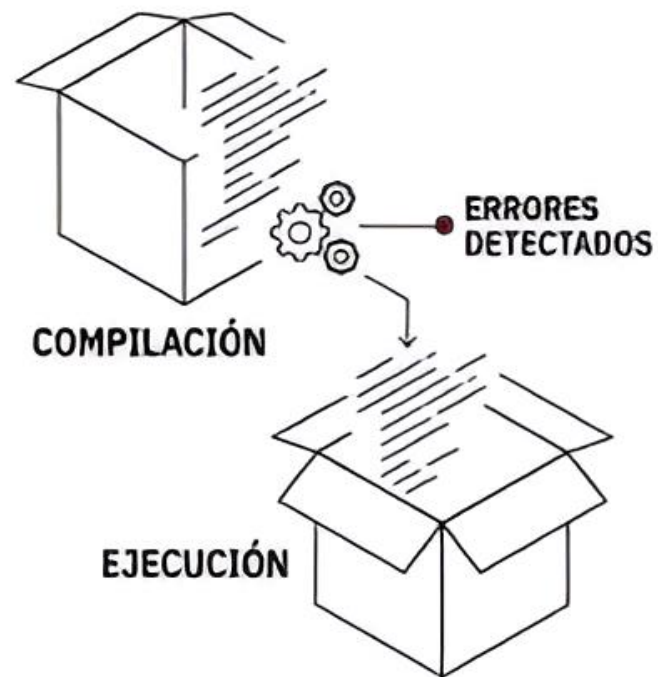
Estructura clave del análisis semántico: un diccionario que el compilador va llenando a medida que analiza el código.

Atributo	¿Qué guarda?	Ejemplo
Nombre	El identificador en sí	contador
Tipo	El tipo de dato	entero, cadena, booleano
Ámbito	Dónde es válido	global, local, argumento
Valor	Valor inicial (si es constante)	10
Ubicación	Dirección de memoria o registro	(lo define el backend)

ANÁLISIS SEMÁNTICO

El análisis semántico está muy ligado al sistema de tipos del lenguaje:

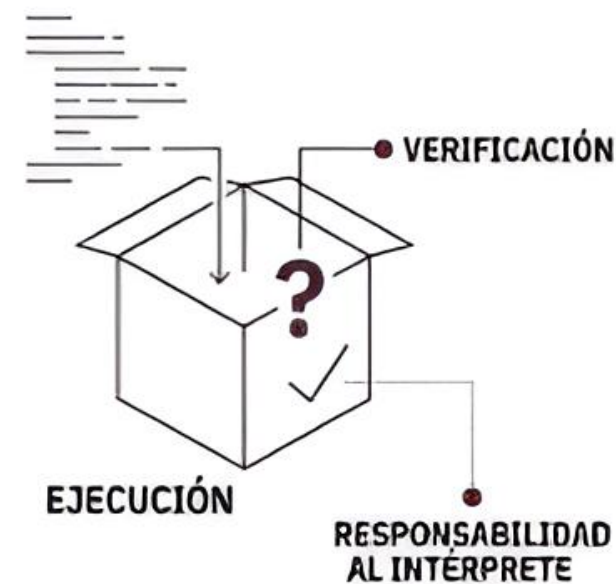
Tipado Estático



Tipos conocidos en compilación, detecta errores antes de ejecutar, análisis semántico riguroso.

C, Java, Rust

Tipado Dinámico



Tipos verificados en ejecución, más flexible, delega responsabilidad al intérprete.

Python, JavaScript, PHP



Los lenguajes con tipado estático realizan un análisis semántico mucho más riguroso: no permiten ambigüedades ni operaciones incoherentes.



Los errores semánticos no dicen que algo esté *mal escrito*, sino que no se puede hacer lo que pides, aunque la sintaxis sea correcta.

ERRORES SEMÁNTICOS: EJEMPLO PRÁCTICO

El análisis semántico recorre el AST anotado y detecta incoherencias de tipo y ámbito. Veamos un ejemplo:

```
INT A = 5;  
INT B = "hola";  
INT C = a + B;
```



A — Correcta

Declarada correctamente como entero con valor.



B — Error semántico

Declarada como entero pero se le asigna un string.



C — Inconsistencia

Intenta sumar entero con string: operación inválida.

ETAPA DE SÍNTESIS: (BACK-END)

Transformar el AST anotado en código que puede ejecutar la máquina



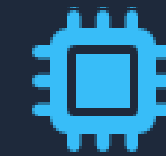
1. Código Intermedio

Traduce el Árbol de Sintaxis Abstracta (AST) a un lenguaje intermedio abstracto (IR). Facilita la portabilidad y separa lógicas.



2. Optimización

Mejora el IR para que el programa sea más veloz y consuma menos recursos sin alterar en absoluto su funcionalidad lógica.



3. Código Objeto

Convierte el IR optimizado en el código máquina o binario que la CPU ejecutará, gestionando registros e instrucciones nativas.

CÓDIGO INTERMEDIO (IR)

Independencia y Portabilidad

El generador de IR establece un "punto medio" más sencillo que el código fuente, independiente de la máquina final.

Beneficios clave:

- Aísla el Front-End de la arquitectura de la CPU.
- Simplifica la aplicación de técnicas de optimización.
- Usan distintos estilos como código de tres direcciones, SSA o el mismo AST
- Utiliza formatos estándar como LLVM IR, código de 3 direcciones o WebAssembly (WASM).



Ejemplos de IR

- LLVM
- WASM
- JAVA Bytecode
- MLIR

OPTIMIZACIÓN DE CÓDIGO

Se busca que el programa final sea más rápido, ligero, o consuma menos energía. Se hacen automáticamente

Independiente de Máquina

Se enfoca en mejorar la estructura algorítmica del IR, aplicable a cualquier CPU.

- **Código muerto:** Se borran líneas inaccesibles o resultados jamás usados.
- **Propagación constante:** Resuelve variables fijas en compilación (ej. $x = 2 * 3$ a $x = 6$).
- **Subexpresiones:** Operaciones idénticas repetidas se calculan solo una vez.

Dependiente de Máquina

Transforma el código para expresar las especificaciones únicas del hardware destino.

- **Instrucciones SIMD:** Procesamiento paralelo de múltiples datos por ciclo.
- **Gestión de registros:** Mantiene los datos críticos cerca de la ALU.
- **Desenrollado de bucles:** Reduce penalizaciones por saltos (branching).

ELIMINACIÓN DE CÓDIGO MUERTO

ORIGINAL

```
int x = 5; int y = 10;  
x = 20; return x;
```

OPTIMIZADO

```
int x = 20; return x;
```

PROPAGACIÓN CONSTANTE

ORIGINAL

```
int a = 5;  
int b = a + 3;
```

OPTIMIZADO

```
int a = 5;  
int b = 8;
```

DESENROLLADO DE BUCLES

ORIGINAL

```
for (int i=0; i<4; i++) {  
    suma += arr[i];  
}
```

OPTIMIZADO

```
suma += arr[0]; suma += arr[1];  
suma += arr[2]; suma += arr[3];
```

SUBEXPRESIONES COMUNES

ORIGINAL

```
int a = (x + y) * z;  
int b = (x + y) * w;
```

OPTIMIZADO

```
int temp = x + y;  
int a = temp * z; int b = temp * w;
```

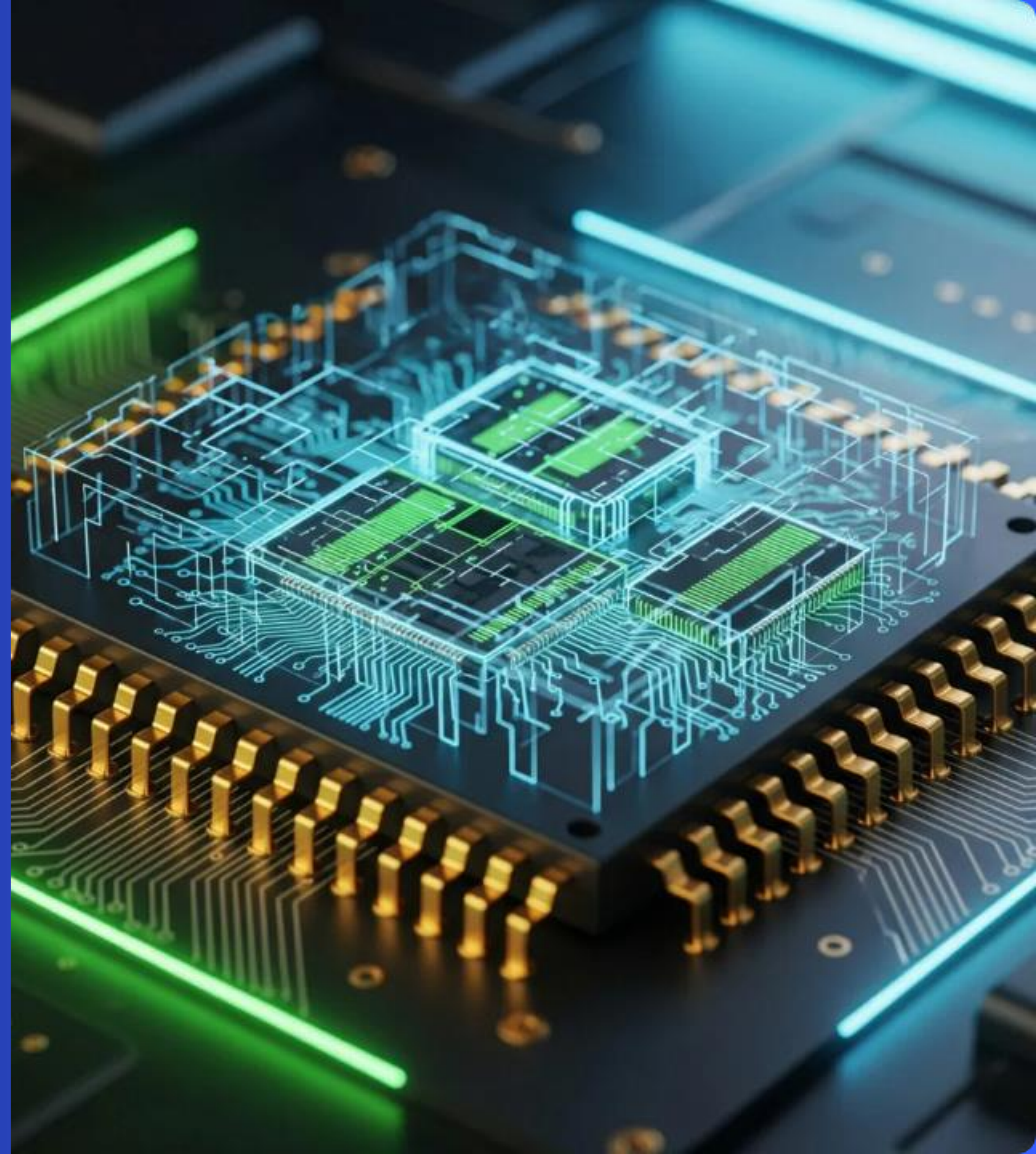

CÓDIGO OBJETO

El destino final: Ensamblador

El IR optimizado se transforma en instrucciones nativas de hardware, un proceso que requiere alta precisión computacional.

Decisiones críticas:

- **Asignación de memoria:** Determinar cuáles variables residen en RAM y cuáles en registros rápidos.
- **Selección:** Mapear operaciones IR a la instrucción más corta/rápida en x86 o ARM.
- **Ordenamiento:** Prevenir esperas forzadas en el pipeline de la CPU.



COMPONENTES TRANSVERSALES

SON COMPONENTES QUE ATRAVIESAN TODO EL COMPILADOR
Y ESTÁN PRESENTES DE INICIO A FIN



Tabla de simbolos

Estructura de datos transversa que el compilador alimenta dinámicamente. Almacena todos los identificadores (variables, funciones, clases) y su metadata.

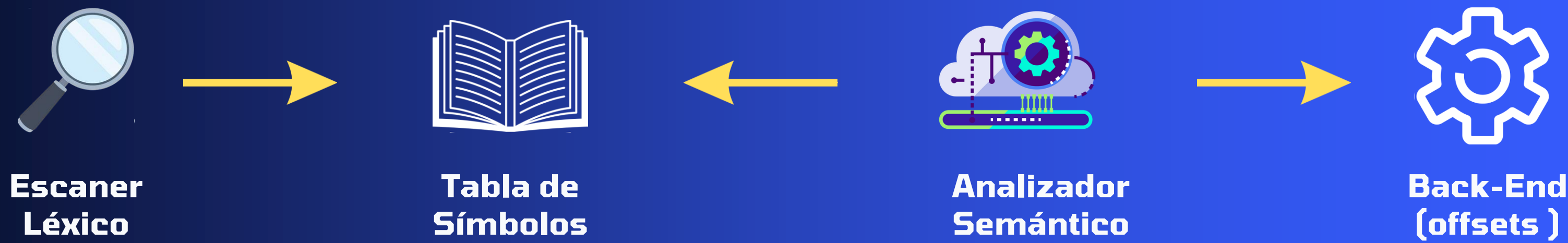


Manejo de errores

Conjunto de estrategias usadas por el compilador para detectar, reportar y recuperarse de errores en el código fuente

LA TABLA DE SÍMBOLOS

Impacto Transversal



Estructura de la Tabla de Símbolos

Atributo	¿Qué información almacena?	Ejemplo Práctico
Nombre	El identificador real del código.	contador, calcularPromedio
Tipo	El tipo de dato primitivo o clase asociada.	int, float, String
Categoría	Clasificación semántica del símbolo.	Variable, constante, función, parámetro
Ámbito (Scope)	Bloque de ejecución donde es válido.	Global, o local a la función main()
Ubicación	Lugar exacto de almacenamiento en memoria.	Registro EAX, offset -8 en la Pila

EJEMPLO

Código fuente:

```
int main() {  
    int a = 5;  
    int b = a + 3;  
    return 0;  
}
```

La tabla de símbolos después del análisis:

Nombre	Tipo	Categoría	Ámbito	Ubicación
main	int	funcion	global	entrada 0x1000
a	int	variable	main	offset -4 (pila)
b	int	variable	main	offset -8 (pila)

Manejo de Errores

Resiliencia en el Compilador

Un compilador debe detectar anomalías, reportarlas con contexto exacto y recuperarse para seguir auditando el código en una sola ejecución.

Estrategias de Recuperación:

- **Modo Pánico:** Descarta tokens hasta encontrar un delimitador seguro (como ; o }).
- **Producción de Error:** Usa reglas gramaticales para anticipar fallos comunes.
- **Corrección Global:** Infiere el cambio mínimo necesario (altamente costoso).



Categorías de Errores



Léxicos

Ocurren cuando el Lexer encuentra caracteres o grafemas ajenos al vocabulario del lenguaje.

```
x = 10 @ 5; // '@' inválido
```



Sintácticos

La secuencia de palabras viola las reglas estructurales de la gramática del lenguaje.

```
if (x > 0 { // Falla en ')'
```



Semánticos

El código es estructuralmente válido, pero su significado u operaciones son incoherentes.

```
int a = "Text"; // Tipo erróneo
```


TENDENCIAS

Compilación Just-In-Time (JIT)



Identificación de código frecuente



Optimización sobre la marcha

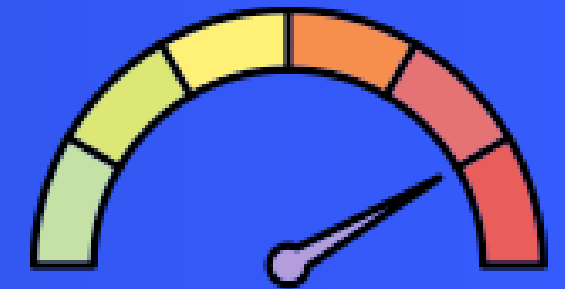


Aceleración de ejecución

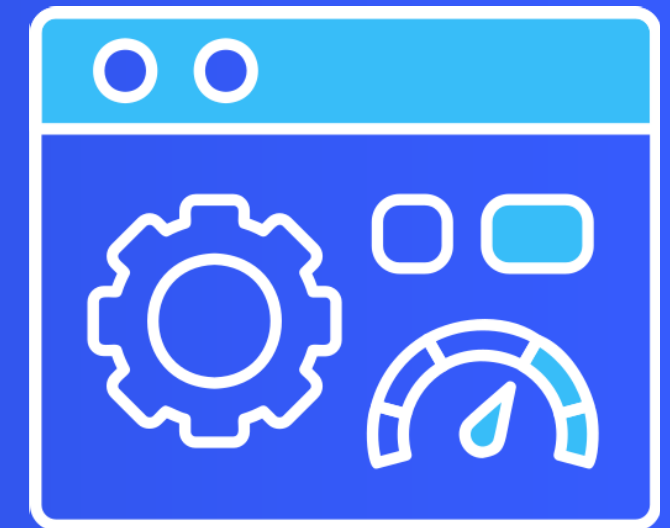
Usado por:



Ventajas



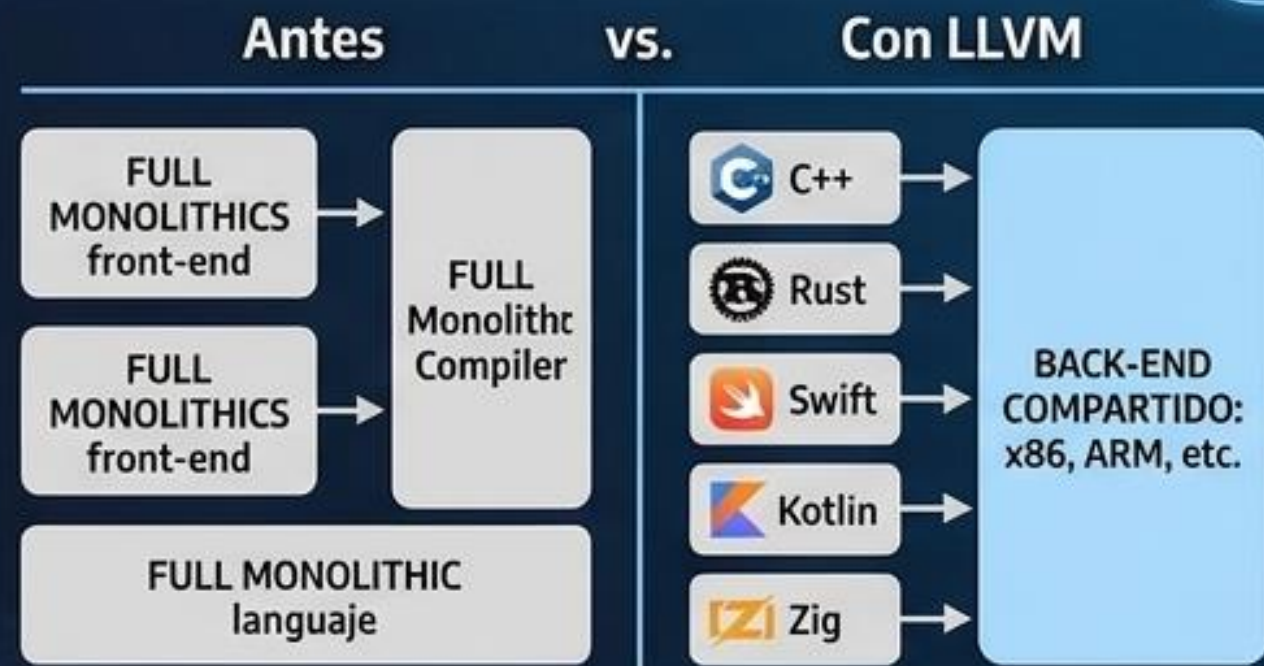
**Rendimiento mejorado:
Acelera rutas calientes**



**Flexibilidad:
Adaptación dinámica**

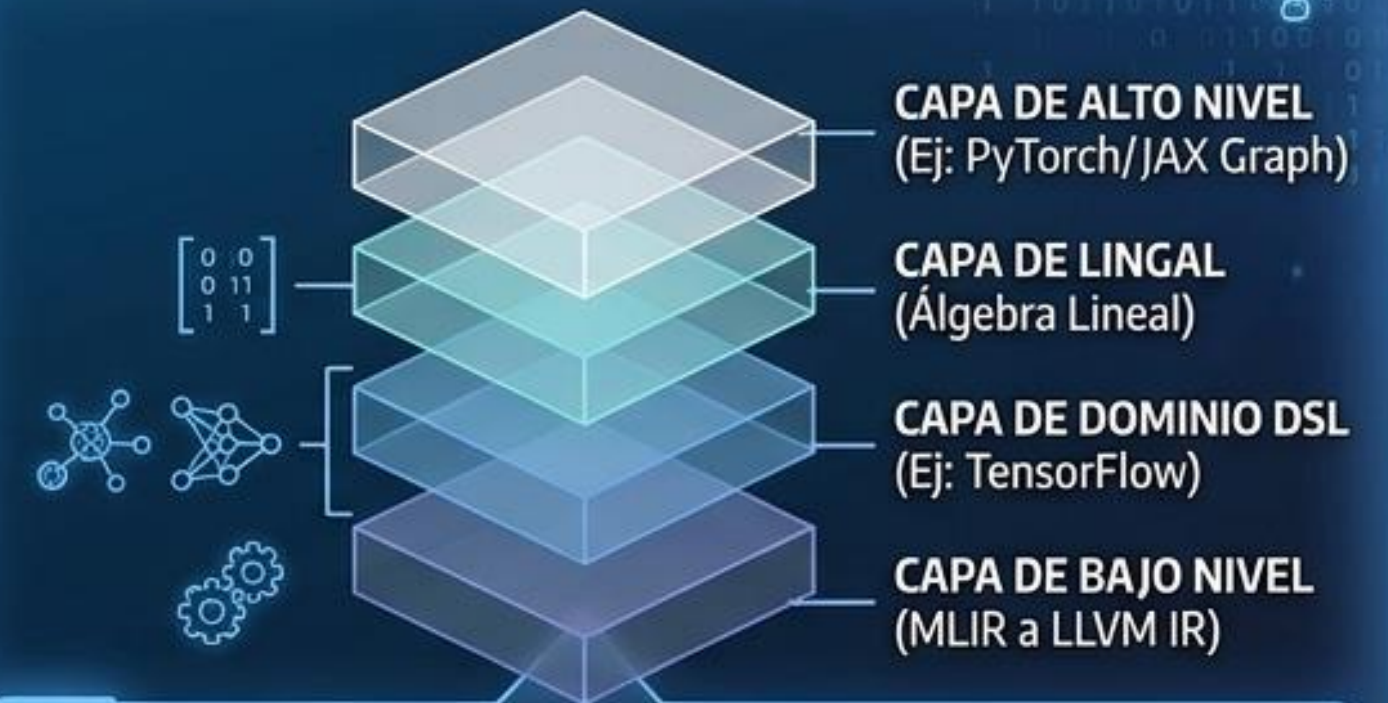
ARQUITECTURAS MODULARES

A. LLVM (LOW LEVEL VIRTUAL MACHINE)



- ✓ Conjunto de bibliotecas reutilizables
- ✓ Cada lenguaje solo necesita su front-end
- ✓ Back-end compartido y optimizado

B. MLIR (MULTI-LEVEL INTERMEDIATE REPRESENTATION)



MLIR para IA y Hardware



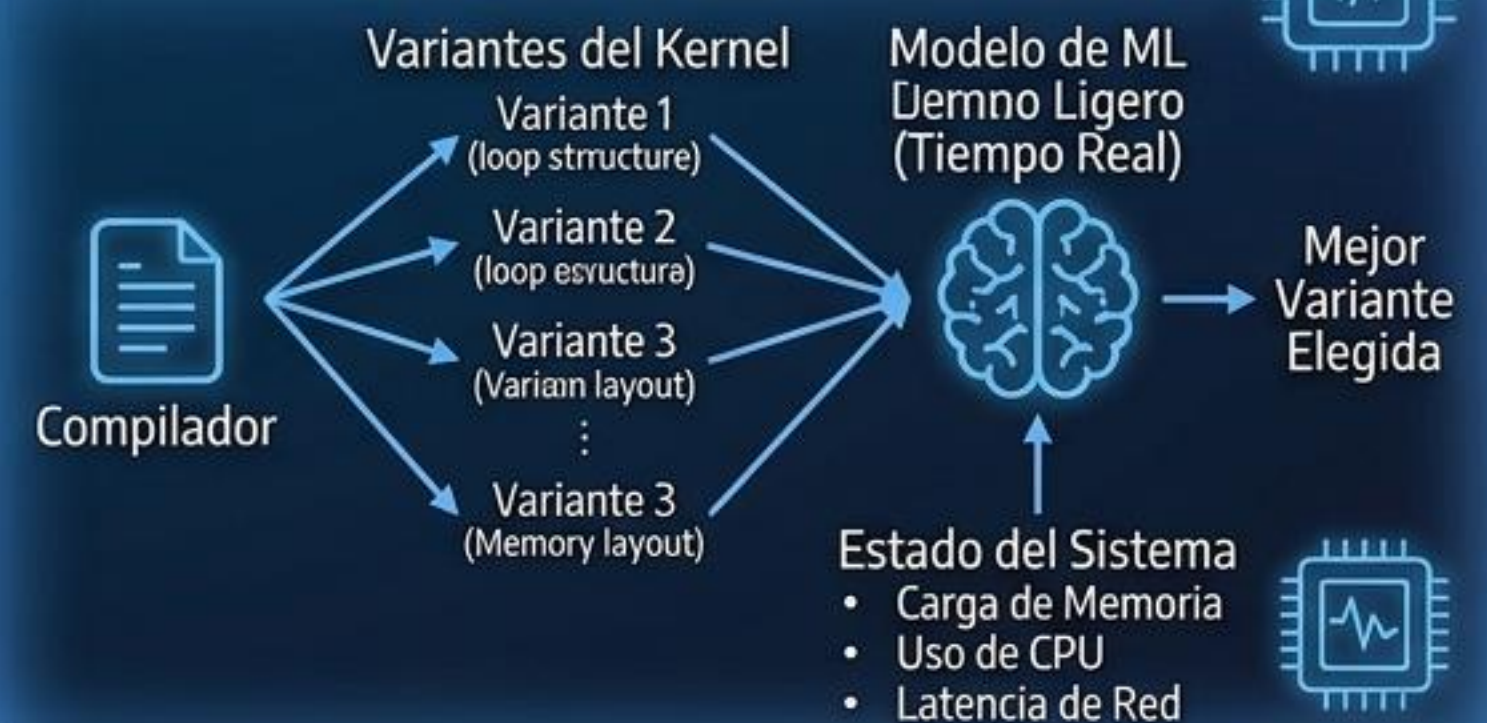
- ✓ Compiladores de IA (TensorFlow, PyTorch, JAX)
- ✓ Lenguajes de Dominio Específico (DSL)
- ✓ Hardware Especializado (TPUs, GPUs)

COMPILACIÓN ASISTIDA POR IA Y RL

A. APRENDIZAJE POR REFUERZO (RL)



B. SISTEMAS CO-DISEÑADOS IA + RUNTIME



WEBASSEMBLY (WASM)

SEGURIDAD Y PORTABILIDAD

1. SANDBOXING



- ✓ Aísla código del sistema anfitrión
- ✓ Ejecución segura, sin acceso a archivos

2. PORTABILIDAD



- ✓ Un solo binario, múltiples plataformas
- ✓ Corre en Windows, Linux, IoT, etc.

3. RENDIMIENTO CASI NATIVO

- ✓ Mucho más rápido que JavaScript
- ✓ Optimizado para el hardware



4. SEGURIDAD DE BYTECODE

- ✓ Validación de bytecode antes de ejecutar
- ✓ Evita errores como buffer overflows



IMPORTANCIA EN LA INGENIERÍA EN INFORMÁTICA

1. MEJOR PROGRAMADOR

- ✓ Entiende optimizaciones y código eficiente



2. TÉCNICAS EN TODAS PARTES

- ✓ Linters, Formateadores, Validadores (JSON/Kubernetes)



3. DISEÑO DE LENGUAJES (DSLs)

- ✓ Base para crear DSLs (ej: SQL, RegEx)



4. CIBERSEGURIDAD

- ✓ Decompiladores (ej: Ghidra) para analizar malware



CONCLUSIONES

1. EVOLUCIÓN CONSTANTE



- ✓ Evolución con IA, WebAssembly y Arquitecturas Modulares

<comp_evol>



2. VALOR FORMATIVO



- ✓ Te convierte en mejor programador
- ✓ Clave en ciberseguridad, DSLs y herramientas



3. MAESTRO FORMAL



"El compilador es el primer maestro formal de esta disciplina."

– Prof. Félix Márquez

<ia_opt_best_v>

**¡GRACIAS
POR SU
ATENCIÓN!**

